

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev, *next;
};

struct Node *head = NULL, *tail = NULL;

void createList(int n) {
    int i, data;
    struct Node *newNode;
    for (i = 1; i <= n; i++) {
        printf("Enter data for node %d: ", i);
        scanf("%d", &data);
        newNode = (struct Node*)malloc(sizeof(struct Node));
        newNode->data = data;
        newNode->prev = newNode->next = NULL;
        if (head == NULL) {
            head = tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }
}

void insertAtFront(int data) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = head;
    if (head == NULL)
        head = tail = newNode;
    else {
        head->prev = newNode;
        head = newNode;
    }
}

void insertAtEnd(int data) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));

```

```

newNode->data = data;
newNode->next = NULL;
newNode->prev = tail;
if (tail == NULL)
    head = tail = newNode;
else {
    tail->next = newNode;
    tail = newNode;
}
}

```

```

void insertAtPosition(int data, int pos) {
    int i;
    struct Node *newNode, *temp = head;
    if (pos == 1) {
        insertAtFront(data);
        return;
    }
    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    for (i = 1; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;
    if (temp == NULL || temp->next == NULL) {
        insertAtEnd(data);
        return;
    }
    newNode->next = temp->next;
    newNode->prev = temp;
    temp->next->prev = newNode;
    temp->next = newNode;
}

```

```

void deleteAtFront() {
    struct Node *temp;
    if (head == NULL) {
        printf("List is empty!\n");
        return;
    }
    temp = head;
    head = head->next;
    if (head != NULL)
        head->prev = NULL;
    else
        tail = NULL;
}

```

```
    free(temp);  
}
```

```
void deleteAtEnd() {  
    struct Node *temp;  
    if (tail == NULL) {  
        printf("List is empty!\n");  
        return;  
    }  
    temp = tail;  
    tail = tail->prev;  
    if (tail != NULL)  
        tail->next = NULL;  
    else  
        head = NULL;  
    free(temp);  
}
```

```
void deleteByValue(int value) {  
    struct Node *temp = head;  
    if (head == NULL) {  
        printf("List is empty!\n");  
        return;  
    }  
    while (temp != NULL && temp->data != value)  
        temp = temp->next;  
    if (temp == NULL) {  
        printf("Value not found!\n");  
        return;  
    }  
    if (temp == head)  
        deleteAtFront();  
    else if (temp == tail)  
        deleteAtEnd();  
    else {  
        temp->prev->next = temp->next;  
        temp->next->prev = temp->prev;  
        free(temp);  
    }  
}
```

```
void display() {  
    struct Node *temp = head;  
    printf("List : ");
```

```
while (temp != NULL) {  
    printf("%d <-> ", temp->data);  
    temp = temp->next;  
}  
printf("NULL\n");  
}
```

```
int main() {  
    int choice, n, data, pos, value;  
  
    printf("Enter number of nodes to create: ");  
    scanf("%d", &n);  
    createList(n);  
  
    while (1) {  
        printf("\n1.Display\n");  
        printf("2.Insert at Front\n");  
        printf("3.Insert at End\n");  
        printf("4.Insert at Position\n");  
        printf("5.Delete at Front\n");  
        printf("6.Delete at End\n");  
        printf("7.Delete by Value\n");  
        printf("8.Exit\n");  
        printf("Enter choice: ");  
        scanf("%d", &choice);  
  
        switch (choice) {  
            case 1:  
                display();  
                break;  
            case 2:  
                printf("Enter data: ");  
                scanf("%d", &data);  
                insertAtFront(data);  
                break;  
            case 3:  
                printf("Enter data: ");  
                scanf("%d", &data);  
                insertAtEnd(data);  
                break;  
            case 4:  
                printf("Enter data and position: ");
```

```
        scanf("%d %d", &data, &pos);
        insertAtPosition(data, pos);
        break;
    case 5:
        deleteAtFront();
        break;
    case 6:
        deleteAtEnd();
        break;
    case 7:
        printf("Enter value to delete: ");
        scanf("%d", &value);
        deleteByValue(value);
        break;
    case 8:
        exit(0);
    default:
        printf("Invalid choice!\n");
    }
}

return 0;
}
```

Enter number of nodes to create: 3

Enter data for node 1: 10

Enter data for node 2: 20

Enter data for node 3: 30

1.Display

2.Insert at Front

3.Insert at End

4.Insert at Position

5.Delete at Front

6.Delete at End

7.Delete by Value

8.Exit

Enter choice: 2

Enter data: 5

1.Display

2.Insert at Front

3.Insert at End

4.Insert at Position

5.Delete at Front

6.Delete at End

7.Delete by Value

8.Exit

Enter choice: 3

Enter data: 40

1.Display

2.Insert at Front

3.Insert at End

4.Insert at Position

5.Delete at Front

6.Delete at End

7.Delete by Value

8.Exit

Enter choice: 4

Enter data and position: 45

3

1.Display

2.Insert at Front

3.Insert at End

4.Insert at Position

5.Delete at Front

6.Delete at End

7.Delete by Value

8.Exit

Enter choice: 5

```
1.Display
2.Insert at Front
3.Insert at End
4.Insert at Position
5.Delete at Front
6.Delete at End
7.Delete by Value
8.Exit
```

Enter choice: 6

```
1.Display
2.Insert at Front
3.Insert at End
4.Insert at Position
5.Delete at Front
6.Delete at End
7.Delete by Value
8.Exit
```

Enter choice: 7

Enter value to delete: 30

```
1.Display
2.Insert at Front
3.Insert at End
4.Insert at Position
5.Delete at Front
6.Delete at End
7.Delete by Value
8.Exit
```

Enter choice: 1

List : 10 <--> 45 <--> 20 <--> NULL

```
1.Display
2.Insert at Front
3.Insert at End
4.Insert at Position
5.Delete at Front
6.Delete at End
7.Delete by Value
8.Exit
```

Enter choice: 8

Process returned 0 (0x0) execution time : 46.523 s

Press any key to continue.

|